

Festival

Nayra está en un festival jugando un juego donde el gran premio es un viaje a Laguna Colorada. El juego se trata de usar fichas para comprar cupones. Ella puede obtener fichas adicionales al comprar un cupón. El objetivo es obtener la mayor cantidad de cupones posible.

Ella comienza el juego con A fichas. Hay N cupones, numerados del 0 al $N - 1$. Nayra tiene que pagar $P[i]$ fichas ($0 \leq i < N$) para comprar el cupón i (y debe tener al menos $P[i]$ fichas antes de la compra). Puede comprar cada cupón como máximo una vez.

Además, cada cupón i ($0 \leq i < N$) tiene un **tipo**, denotado por $T[i]$, que es un número entero **entre 1 y 4, inclusive**. Después que Nayra compre el cupón i , el número restante de fichas que tiene se multiplica por $T[i]$. Formalmente, si tiene X fichas en algún momento del juego, y compra el cupón i (que requiere $X \geq P[i]$), entonces tendrá $(X - P[i]) \cdot T[i]$ fichas después de la compra.

Tu tarea es determinar qué cupones debe comprar Nayra y en qué orden, para maximizar el número total de **cupones** que tiene al final. Si hay más de una secuencia de compras que logre esto, puedes reportar cualquiera de ellas.

Detalles de Implementación

Debes implementar la siguiente función.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : el número inicial de fichas que tiene Nayra.
- P : un arreglo de longitud N que especifica los precios de los cupones.
- T : un arreglo de longitud N que especifica los tipos de cupones.
- Esta función se llama exactamente una vez para cada caso de prueba.

La función debe devolver un arreglo R , que especifica las compras de Nayra de la siguiente manera:

- La longitud de R debe ser el número máximo de cupones que Nayra puede comprar.
- Los elementos del arreglo son los índices de los cupones que debe comprar, en orden cronológico. Es decir, compra primero el cupón $R[0]$, luego el cupón $R[1]$, y así sucesivamente.

- No debe haber elementos repetidos en R .

Si no se puede comprar ningún cupón, R debe ser un arreglo vacío.

Restricciones

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ para cada i tal que $0 \leq i < N$.

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	5	$T[i] = 1$ para cada i tal que $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
3	12	$T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra puede comprar todos los N cupones (en algún orden).
6	16	$(A - P[i]) \cdot T[i] < A$ para cada i tal que $0 \leq i < N$.
7	18	Sin restricciones adicionales.

Ejemplos

Ejemplo 1

Considera la siguiente llamada:

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra tiene inicialmente $A = 13$ fichas. Ella puede comprar 3 cupones siguiendo el orden mostrado a continuación:

Cupón comprado	Precio del cupón	Tipo de cupón	Número de fichas después de la compra
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

En este ejemplo, no es posible que Nayra compre más de 3 cupones, y la secuencia de compras descrita anteriormente es la única forma en que puede comprar 3 de ellos. Por lo tanto, la función debería devolver $[2, 3, 0]$.

Ejemplo 2

Considera la siguiente llamada:

```
max_coupons(9, [6, 5], [2, 3])
```

En este ejemplo, Nayra puede comprar ambos cupones en cualquier orden. Por lo tanto, la función debe devolver $[0, 1]$ o $[1, 0]$.

Ejemplo 3

Considera la siguiente llamada:

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

En este ejemplo, Nayra tiene una ficha, que no es suficiente para comprar ningún cupón. Por lo tanto, la función debería devolver $[]$ (un arreglo vacío).

Calificador local

Formato de entrada:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de salida:

```
S
R[0] R[1] ... R[S-1]
```

Aquí, S es la longitud del arreglo R devuelto por `max_coupons`.